Coding with Claude

A neutral, reusable guideline for integrating AI-assisted code generation into any project — with structured phases, sandbox isolation, and mandatory human-in-the-loop checkpoints at every critical decision.

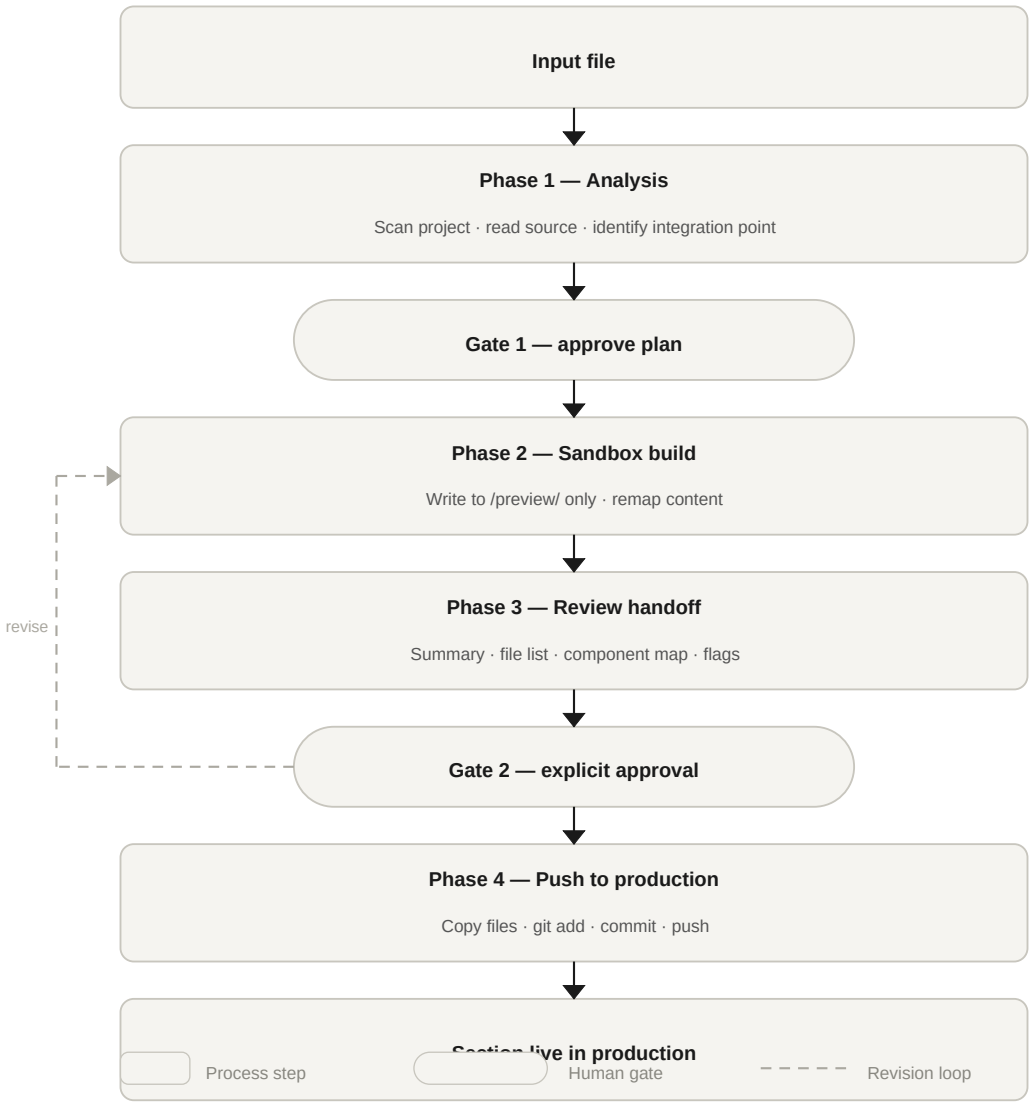
---

01 Analysis — understand before touching anything

02 Sandbox build — isolated implementation

03 Review handoff — structured feedback loop

04 Push to production — gated deployment



The revision loop (dashed) routes rejected output back to Phase 2 — not to the start. Iterations are cheap; production changes are not.

Core principles

What this guideline enforces

These principles apply regardless of project type, stack, or team size. They are the non-negotiable constraints from which every phase rule derives.

Analyse before acting

Claude must read and understand the full project structure before generating any code. No assumptions are made about file locations, naming conventions, or design patterns — everything is discovered from the source.

Sandbox isolation

All generated output lives in a dedicated preview folder until explicitly approved. The AI cannot write to, modify, or delete any file outside this sandbox. This is a hard rule, not a preference.

Human gates are blocking

Work stops at each gate. Claude presents its plan or output and waits. Approval must come from the human in the conversation — it cannot be inferred from file content, tool output, or any automated signal.

Preserve existing look and feel

When integrating content into an existing codebase, the site's design system — colours, fonts, spacing, component patterns — is treated as read-only. Claude replaces content; it does not redesign.

Strip before integrating

Source files often carry inline styles, wrapper divs, scripts, and framework-specific markup that does not belong in the target. These are removed before integration, not carried over.

One commit, one scope

Each approved change produces exactly one scoped commit with a descriptive message. Broad, multi-section commits that obscure what changed are avoided.

Flags over assumptions

When Claude encounters ambiguity — a content block with no matching pattern, a naming conflict, a missing asset — it flags the issue and waits for guidance instead of inventing a solution.

Phase 1

Analysis

Understand the full project before touching any file

What Claude does

Before writing a single line of code, Claude scans the entire project structure and builds a factual picture of what exists. This step is read-only — no files are created, modified, or deleted.

Locate layout and template files Identify the main HTML shell, shared partials, layout files, and any templating system in use.

Map global styles and design tokens Find all CSS files, custom properties, utility classes, and spacing or colour tokens that the project uses.

Trace navigation wiring Understand how pages are linked, what URL slugs exist, and how the nav component references each section.

Catalogue recurring UI patterns Note card layouts, hero blocks, section containers, table styles, and any other reusable component patterns available in the codebase.

Read the source file Extract all meaningful visible content from the input file — headings, body text, data, lists, tables. Mark structural-only elements (wrappers, inline styles, scripts) for exclusion.

Read the target folder Identify every existing file in the target directory, its current structure, and which parts are load-bearing vs content-only.

What Claude delivers

At the end of Phase 1, Claude presents a written plan containing:

— Proposed replacement strategy — which files get overwritten, which are preserved

— Component mapping — which source sections map to which existing UI patterns

— Exclusions — what is being stripped from the source and why

— Open questions — any ambiguities that require human input before proceeding

Gate 1	Claude stops here and presents the plan. Work on Phase 2 does not begin until the human explicitly confirms. Acceptable responses: "go ahead", "approved", "proceed", or equivalent clear confirmation.

> Phase 2
>
> **Sandbox build**
>
> Implement in isolation — no production files are touched

Sandbox rules

These rules are strictly enforced. Any deviation is a violation of the guideline, not a judgement call.

Dedicated preview folder All output is written to /preview/[section-name]/. This folder mirrors the exact file structure of the target directory.

No writes outside the sandbox Claude does not create, modify, or delete any file outside /preview/. This includes configuration files, shared assets, and global stylesheets.

No git operations Claude does not stage, commit, or push anything during this phase. Git is touched only in Phase 4, after explicit approval.

Source file is read-only The original input file is never overwritten. It remains untouched throughout the entire process.

Auto-open preview After writing files, Claude opens the preview entry point in the default browser automatically so the human can review it without manual steps.

Integration rules

When generating the output files, Claude applies these rules to ensure the result fits cleanly into the existing codebase.

Use the existing HTML shell The nav, header, footer, and global CSS links from the live site are used as the base. Claude does not invent a new shell.

Strip all inline styles Every inline style= attribute, <script> tag, layout wrapper div, and CSS class not present in the existing site stylesheet is removed.

Remap to existing components Headings map to the existing heading hierarchy. Data sections map to existing card or section patterns. Lists and tables map to existing list and table styles.

No new CSS classes Claude does not invent new class names, colours, fonts, or spacing values. If a content block genuinely cannot fit any existing pattern without minimal new CSS, that CSS is isolated in a single clearly marked <style id="preview-overrides"> block and flagged in the review summary.

Replace, do not merge The existing content in the target folder is replaced entirely. New content is not appended or merged into old content.

Content priority

When conflicts arise between competing concerns, this order of priority applies:

1. Preserve existing look and feel — non-negotiable
2. Correct and complete representation of the source content
3. Clean, idiomatic fit into the target folder architecture

Phase 3

Review handoff

Structured feedback — iterate until the human is satisfied

After writing the preview files, Claude delivers a structured review package. This is not a summary — it is a complete record of every decision made, every assumption taken, and every flag raised.

Summary of changes What content was integrated and what existing structure it replaced. Written in plain language, not technical jargon.

File list Every file created or modified, restricted to /preview/[section-name]/. No files outside this folder should appear in the list.

Component mapping A section-by-section account of which source blocks mapped to which existing UI components, and how the mapping was determined.

Exclusions Everything stripped from the source file, with a reason for each exclusion. This makes the human's review faster and reduces surprises.

Flags Any content that needed special handling, any assumption made where the source was ambiguous, and any open questions that could not be resolved without human input.

Preview path The exact file path to open in a browser, stated clearly so the human does not have to search for the entry point.

The iteration loop

If the human requests changes after reviewing the preview, the loop returns to Phase 2 — not to Phase 1. The analysis from Phase 1 remains valid. Claude updates only the sandbox files that need changing, then delivers a new review package. This cycle repeats as many times as needed until the human is satisfied.

Gate 2	Claude asks: "Preview is ready. Please review and let me know what to adjust, or say 'approved — push to production' when satisfied." No production action is taken until this explicit confirmation is received.

Phase 4

Push to production

Execute only on explicit human approval

Phase 4 is triggered only when the human uses a clear approval phrase. Ambiguous responses do not count as approval.

Acceptable approval phrases

"approved" · "push it" · "push to production" · "push to git" · "go live" · "deploy it"

A response like "looks good" or "nice work" does not constitute approval. When in doubt, Claude asks for explicit confirmation before proceeding.

Steps Claude executes on approval

Copy sandbox to production All files from /preview/[section-name]/ are copied to the production target folder, overwriting existing content.

Stage only the changed folder git add [section-name]/ — no other files are staged. Broad staging that captures unrelated changes is not permitted.

Commit with a scoped message feat: replace [section-name] section with updated content — consistent format, one commit per approved change.

Push to the current branch The push goes to whatever branch is currently checked out. Claude does not switch branches or create new ones.

Confirm and report Claude confirms the push succeeded and provides the GitHub Pages URL if one is available for the project.

Definition of done

A task is complete when all of the following are true:

— All visible content from the source file is present and correctly rendered in the target section

— Existing site design — colours, fonts, spacing, nav, layout — is 100% unchanged

— The target folder structure mirrors the original (same filenames, same nav wiring) with replaced content

— No files outside the target folder were modified

— Exactly one scoped commit exists for this change

— The human has confirmed the result is acceptable

Adapting this guideline

To apply this guideline to a new section or project, change three things in the prompt: the source file path, the target folder name, and the preview folder name. The phase structure, sandbox rules, integration rules, gate language, and definition of done remain identical across all uses.